Rama • Rediseño Funcional de SIGESTA

https://gemini.google.com/app/56d57baebd8a27d4

User prompt: hablemos de programación funcional, Python interfaces graficas y un pdf que te voy a pasar. Yo tengo un proyecto antiguo que realice en C++. es un gestor de tareas, que se conectaba a una base de datos y podías compartir las tareas con ciertos usuarios. Quiero que me digas porque es mala idea: crees que sea buena idea crear una versión de SIGESTA este proyecto de mi materia programación funcional, crees que sea posible implementar todos los requisitos.

Response: Replicar **SIGESTA** tal como fue concebido en C++ y MySQL dentro de esta práctica de Programación Funcional es una mala idea si intentas arrastrar su arquitectura original. Sin embargo, **es completamente viable si adaptas el dominio** (gestor de tareas) a los lineamientos estrictos de la guía.

La guía lista explícitamente el **"gestor de tareas"** como un dominio válido. El problema real no es la idea de gestionar tareas, sino el choque conceptual entre tu implementación anterior y lo que evalúa la rúbrica.

Por qué arrastrar el SIGESTA original es una mala idea

1. **La base de datos relacional asfixia el paradigma funcional:**
    - En tu código C++, casi toda la lógica son consultas `INSERT`, `UPDATE` y `SELECT` directas a MySQL.
    - En programación funcional para esta materia, el foco está en **estructuras de datos en memoria**, inmutabilidad y transformaciones puras.
    - La rúbrica penaliza severamente declarar funciones como puras si hacen acceso a I/O, base de datos o mutaciones. Si dependes de MySQL para filtros, joins y reportes, dejas sin espacio a `map`, `filter`, `reduce` y a las funciones de orden superior en Python.
2. **Riesgo crítico de acoplamiento UI-Lógica:**
    - Tu proyecto en C++ mezcla en el mismo flujo la lectura por consola, las consultas a base de datos y la navegación de menús.
    - La guía impone como **regla crítica** una separación estricta: Capa visual (`tkinter` + `ttkbootstrap`), Capa lógica (reglas y transformaciones puras) y Capa de estado. Concentrar lógica dentro de los callbacks de botones está explícitamente penalizado.
3. **Sobrecarga innecesaria de alcance (Scope Creep):**
    - SIGESTA incluía autenticación de usuarios, roles de admin, envío/recepción de tareas compartidas y persistencia relacional.
    - La guía exige entregables pesados de fundamentación: `analisis-funcional.md`, `bitacora-ia.md` y un video justificando cada concepto (closure, pureza, inmutabilidad, pattern matching). Gastar tiempo configurando drivers de MySQL o sesiones de usuario no suma puntos en los 12 puntos de evaluación funcional (Unidades 1 y 2).

Cómo es posible implementar todos los requisitos con este dominio

Si decides mantener el concepto de **gestor de tareas**, debes reformularlo como un **"Simulador/Gestor Funcional de Tareas"** en memoria:

- **Modelado de datos (Unidad 1):** Usar `NamedTuple` o `@dataclass(frozen=True)` para modelar `Tarea`, `Categoria` y `Usuario`. Las prioridades o estados pueden definirse con `Union` o tipos literales.
- **Pattern Matching (Unidad 1):** Usar `match/case` para un despachador de acciones (ej. `match accion:` procesar `CrearTarea`, `CompletarTarea`, `FiltrarPorPrioridad`, `CompartirConUsuario`).
- **Inmutabilidad y Estado (Unidad 2):** El estado global de las tareas se actualiza devolviendo una nueva colección/tupla de tareas sin mutar la lista original.
- **Pureza y Funciones de Orden Superior (Unidades 1 y 2):**
  - **3 funciones puras:** Funciones que reciban la lista de tareas y devuelvan estadísticas (ej. porcentaje de avance, cálculo de métricas), transformaciones de texto o validaciones.
  - **2 funciones impuras:** Las funciones que interactúan con los widgets de `ttkbootstrap` o las que leen/escriben un archivo de exportación/reporte.
  - **Uso de `filter`, `map`, `reduce`:** Filtrar tareas por categoría o estado con `filter`, proyectar títulos/resúmenes con `map`, y consolidar estadísticas con `reduce` en lugar de hacerlo mediante sentencias SQL.
- **Closure real (Unidad 2):** Crear fábricas de filtros (por ejemplo, una función `crear_filtro(criterio, valor)` que devuelva una función cerrando el entorno léxico para filtrar dinámicamente según prioridad o categoría).

El dominio de SIGESTA encaja con los requisitos de la actividad siempre que **elimines MySQL**, mantengas el estado inmutable en memoria y muevas toda la lógica de cálculo y filtrado a funciones puras de Python.

¿Prefieres enfocar la entrega en adaptar SIGESTA como un gestor de tareas funcional en memoria, o te inclinas más por la propuesta de la arena/simulador que sugiere la guía?

User prompt: dejando de lado el codigo de c++ (lo realice en 2do semestre), entiendo que el proyecto debe usar una estrutra clara algo como paquetes en java o mejor dicho ser un monolito modular. mi duda es: No es posible hacer o usar sentencias SQL en python usando lambdas o funciones de orden superior o eso complica de mas el proyecto.

Response: Técnicamente **sí es posible** usar sentencias SQL en Python combinándolas con funciones de orden superior o lambdas, pero **para esta entrega complica el proyecto de forma innecesaria y aumenta el riesgo de ser penalizado**.

Aquí está el análisis técnico de por qué ocurre esto:

1. ¿Cómo se vería combinar SQL con orden superior?

Podrías construir abstracciones funcionales alrededor de la base de datos, por ejemplo:

- Una función de orden superior que reciba una conexión o transacción y tome como argumento una función pura que procese los resultados (un mapper funcional de filas).
- Un closure que construya dinámicamente cláusulas o filtros SQL basados en parámetros.
- Funciones de orden superior para manejar el ciclo de vida de la conexión (tipo *currying* o *context wrappers*).

2. Por qué complica de más el proyecto y choca con la guía

1. La I/O destruye la pureza referencial:
    - Toda consulta a base de datos (`cursor.execute`, `cursor.fetchall`) es un **efecto secundario** (I/O externa). Depende del estado de la base de datos y puede fallar por problemas de red o bloqueos.
    - La guía exige **al menos 3 funciones puras justificadas** y penaliza severamente declarar funciones como puras si hacen acceso a archivos, I/O o mutación externa. Si la manipulación de datos se hace mediante SQL, terminas empujando la lógica a funciones impuras o tendrás que hacer malabares para separar qué parte es cálculo puro y qué parte es persistencia.
2. SQL ya compite contra `map`, `filter` y `reduce`:
    - El trabajo que normalmente hacen `filter()` y `map()` en programación funcional en memoria, en una base de datos lo resuelve el motor con `WHERE` y `SELECT columna1, columna2`.
    - Si filtras con SQL, usar `filter()` en Python después se convierte en un filtro redundante o artificial, lo cual cae en la penalización de: *"se usaron lambdas solo para parecer funcionales"* o *"el proyecto se ve bonito, pero conceptualmente está vacío"*.
3. Inmutabilidad comprometida:
    - Una base de datos relacional es intrínsecamente mutable (haces `UPDATE` y `DELETE` sobre registros existentes).
    - La rúbrica exige demostrar **inmutabilidad real** al transformar el estado, no mutar entidades directamente y justificarlo en el video y en el documento de análisis. Modelar esto en memoria con colecciones inmutables (`tuple`, `frozenset`) y tipos de datos inmutables (`NamedTuple` o `dataclass(frozen=True)`) es directo, limpio y fácil de defender técnicamente frente al docente.
4. **Complejidad arquitectónica extra:**
    - En lugar de concentrarte en el diseño de un monolito modular limpio (con capas bien delimitadas: vista, lógica funcional pura, estado inmutable), tendrías que gestionar drivers (`sqlite3` o `mysql-connector`), manejo de cursores, esquemas y manejo de excepciones de base de datos.
    - Recuerda que la rúbrica evalúa la comprensión de los conceptos de las Unidades 1 y 2 (12 de los 25 puntos). Ningún punto evalúa conectividad ni persistencia relacional.

Enfoque recomendado: Monolito modular en memoria

Lo ideal para este proyecto es mantener el dominio del gestor de tareas pero manejar la información en **memoria**:

- **Módulo de modelos (`models.py`):** `dataclass(frozen=True)` o `NamedTuple` para definir `Tarea`, `Usuario`, `Categoria`.
- **Módulo de lógica pura (`logic.py`):** Funciones puras que reciben colecciones inmutables y devuelven nuevas colecciones o métricas usando `map`, `filter`, `reduce`, `match/case` y closures para generar filtros dinámicos.
- **Módulo de estado (`state.py`):** Manejo inmutable del estado global de la aplicación.
- **Módulo de interfaz (`ui.py`):** Widgets de `tkinter` / `ttkbootstrap` que actúan como la capa impura encargada de renderizar y disparar eventos hacia la lógica.

Si quieres persistencia sin complicarte ni meter SQL, puedes dejar una función impura que serialice y deserialice el estado hacia un archivo local (`JSON` o texto) al abrir o guardar, cumpliendo limpiamente con el requisito de *"funciones impuras justificadas por efectos secundarios de lectura/guardado"*.

User prompt: bien es que dejarlo en memoria siento que no tendria sentido, uso me gestor de tareas, pero cuando lo cierro y lo vuelvo abrir no están mis tareas. bien entonces podemos solucionar eso con el json para la parte de presistencia. ahora darle una razon: una empresa que tiene que gestionar la tareas de sus trabajadores. y se comunican mediante el software de sigesta de forma local y conectados en red. tiene sentido?. Obvio que en un entorno real deberia exisitir algun tipo de proteccion en la compu que sirva de servidor. para que la gente no lo pueda modificar. pero eso se va de la materia solo quiero darle una historia y una razon de ser al proyecto.

Response: Tiene total sentido y es una excelente justificación narrativa para el proyecto. De hecho, encaja con los requerimientos técnicos y conceptuales de la rúbrica.

Al plantearlo de esa forma, resuelves el problema de la persistencia y al mismo tiempo cumples con las reglas de diseño obligatorias de la guía:

1. La frontera clara entre lo Puro y lo Impuro

En programación funcional existe un patrón clave: **Functional Core, Imperative Shell** (Núcleo funcional, caparazón imperativo).

- **El caparazón impuro (Efectos secundarios):** Cargar el archivo JSON al iniciar la aplicación y guardarlo al salir (o al presionar un botón de guardar/sincronizar) son **efectos secundarios reales de I/O**. Esto te cubre de inmediato el requisito de *al menos 2 funciones impuras justificadas*.
- **El núcleo funcional (Puro):** Una vez que los datos del JSON se leen y se transforman en estructuras inmutables en memoria (`dataclass(frozen=True)` o `NamedTuple`), **todo el procesamiento de las tareas ocurre en funciones puras**:
  - Asignación o reasignación de tareas a un trabajador.
  - Cálculo de carga laboral o métricas del equipo.
  - Filtrado de tareas pendientes o urgentes.

2. Cómo encaja la historia en el documento de análisis

En el archivo obligatorio `analisis-funcional.md` te piden explicar el *problema que resuelve la aplicación* y *qué partes son puras/impuras*:

- **El problema:** Una pyme o equipo operativo que necesita coordinar flujos de trabajo locales sin depender de infraestructuras complejas en la nube, usando una estación de trabajo que centraliza el estado del equipo.
- **Justificación de la persistencia:** Explicas técnicamente que la persistencia en JSON aísla la I/O (disco/red local) en las fronteras de la aplicación, permitiendo que la lógica de negocio permanezca determinista, testeable y libre de efectos colaterales.

3. Ideas de casos de uso para lucir los requisitos funcionales

Con esta narrativa de gestión empresarial/equipo de trabajo, los requisitos de la Unidad 1 y 2 se justifican de manera natural:

- **Closure real (Unidad 2):** Un generador de filtros de asignación. Por ejemplo, una función `crear_filtro_trabajador(id_trabajador)` o `crear_filtro_area(departamento)` que capture el parámetro y devuelva una función predicado para usar dentro de `filter()`.
- **Pattern Matching con `match/case` (Unidad 1):** Un despachador de eventos de la empresa (ej. `match evento: AsignarTarea(id_tarea, id_usuario)`, `CompletarTarea(id_tarea)`, `CambiarPrioridad(id_tarea, nueva_prioridad)`).
- **Uso de `map`, `filter`, `reduce` (Unidad 2):**
  - `filter`: Obtener solo las tareas de alta prioridad o pendientes de un trabajador.
  - `map`: Proyectar resúmenes de texto o tarjetas formateadas para la interfaz gráfica.
  - `reduce`: Calcular métricas acumuladas (por ejemplo, horas estimadas totales del equipo o total de tareas por estado).

La narrativa funciona bien y te evita lidiar con la complejidad de un motor SQL, manteniéndote alineado con lo que evalúa la rúbrica.

Si te parece bien esta base, podemos planificar la estructura de módulos (capas) o definir las estructuras de datos inmutables para el dominio. ¿Deseas que empecemos a diseñar esa estructura modular? *(Recuerda que si en algún momento necesitas ejemplos de código, me confirmas para dártelos con tu visto bueno).*

User prompt: quiero comenzar por algo mas simple redactar el problema a detalle y la idea de la solución. una pequeña empresa, el jefe quiere designar tareas a cada trabajador, cada trabajaro puede cambiar el estado de la tarea (hecho, en proceso, no iniciado) y tambien puede escribir sus propias tareas. asi el jefe le puede decir a juan, tienes 3 tareas y juan para organizar su dia tiene que hacer 2 cosas mas y en total el tiene 5. seria mas de un json para usuario y otro para las tareas como seria bueno plantearlo.

Response: Aquí tienes la redacción estructurada del problema y la solución (muy útil para la sección obligatoria del `analisis-funcional.md`), junto con la propuesta de diseño para los

archivos JSON:

1. Redacción del Problema

Contexto:

En una pequeña empresa u oficina operativa, la coordinación diaria entre la administración (el jefe/líder de equipo) y los colaboradores (trabajadores) suele realizarse mediante canales dispersos (mensajes de texto, notas en papel o instrucciones verbales), lo que provoca falta de seguimiento y duplicidad de esfuerzos.

Problema puntual:

- El líder del equipo necesita asignar tareas directas a cada colaborador y supervisar su avance en tiempo real.
- El colaborador necesita gestionar no solo lo que le asigna su líder, sino también sus propias subtareas o compromisos operativos diarios para organizar su jornada de trabajo de forma integral.
- Al carecer de una herramienta centralizada y simple, se pierde visibilidad sobre la carga real de trabajo de cada persona (por ejemplo, saber que un colaborador tiene 3 tareas asignadas por el líder más 2 personales, totalizando 5 pendientes).

2. Idea de la Solución (SIGESTA Funcional)

Propuesta:

Desarrollar una aplicación de escritorio ligera basada en **Python (`tkinter` + `ttkbootstrap`)** que gestione el flujo de tareas bajo un modelo de datos inmutable y funciones puras.

Dinámica del sistema:

1. **Roles y perfiles simples:** La aplicación permite seleccionar el usuario activo (Jefe o Trabajadores específicos como Juan, María, etc.).
2. **Origen de la tarea:** Cada tarea cuenta con un tipo de origen:
    - *Asignada por líder:* Creada por el jefe para un trabajador específico.
    - *Personal / Autogestionada:* Creada directamente por el trabajador para su propio día.
3. **Flujo de estados:** Cada trabajador puede actualizar el estado de sus tareas entre:
    - `NO_INICIADO`
    - `EN_PROCESO`
    - `HECHO`
4. **Visibilidad y métricas:**
    - El trabajador ve su lista consolidada de la jornada (asignadas + propias).
    - El jefe puede alternar vistas, filtrar la carga global por trabajador mediante funciones de orden superior (`filter`) y ver estadísticas acumuladas usando `reduce`.

3. Planteamiento de los JSON: ¿Un solo archivo o varios?

Para un proyecto con arquitectura funcional y persistencia simple en archivo, **la mejor opción técnica es usar un único archivo JSON que contenga todo el estado (`datos.json` o `estado_sistema.json`).**

¿Por qué un solo JSON es superior para este proyecto?

1. Facilita la inmutabilidad atómica:

   En programación funcional, el estado de la aplicación se puede representar como una única estructura compuesta (por ejemplo, una tupla o registro `EstadoApp(usuarios=..., tareas=...)`). Al guardar, se serializa todo ese estado de una sola vez. Al leer, se carga completo y se pasa a la lógica pura.

2. **Evita inconsistencias de sincronización:**

   Si manejas dos archivos (`usuarios.json` y `tareas.json`), necesitas dos operaciones de I/O separadas. Si una falla o una tarea apunta a un usuario que no terminó de sincronizarse, creas estados inconsistentes.

3. **Estructura limpia:** En un solo archivo mantienes la lista de usuarios y la lista de tareas relacionadas simplemente por el identificador del usuario.

Estructura conceptual del JSON:

- **`usuarios`**: Lista con objetos de usuario (ej. ID, nombre, rol como `"ADMIN"` o `"EMPLEADO"`).
- **`tareas`**: Lista con objetos de tarea (ej. ID, título, descripción, `creador_id`, `asignado_id`, prioridad, estado).

De este modo:

- Cuando Juan entra a la app, una función pura filtra:

$$\text{tareas de Juan} = \{t \in \text{tareas} \mid t.\text{asignado_id} == \text{id_juan}\}$$

- Puede ver con claridad cuáles fueron creadas por él mismo (`creador_id == id_juan`) y cuáles le fueron asignadas por su jefe (`creador_id != id_juan`).

¿Te convence este enfoque de un solo archivo de estado para la persistencia, o tienes alguna preferencia de por qué te gustaría separarlo en dos?
